

Blackpebble — Replit Prompt: Live Stats, CA Bar, Pie Chart & Wallet Connect

Add the following features to the existing Blackpebble website. Keep all existing pages, design, and styling intact. The site uses a dark theme (#0a0a0a background), gold accent (#c9a96e), Playfair Display for headings, Inter for body text. All new elements must match this aesthetic perfectly.

1. CONTRACT ADDRESS BAR (All Pages)

Add a slim, fixed bar at the VERY TOP of every page — above the navigation header. This bar stays visible on scroll.

Specs:

- Height: 36px
- Background: #0d0d0d (slightly lighter than main bg to create subtle separation)
- Border-bottom: 1px solid #1a1a1a
- Content centered horizontally
- Layout: "\$BLK" label (gold color, bold) + contract address in monospace font (white, slightly smaller) + copy icon button

HTML/CSS concept:

Plain Text

```
<div class="ca-bar">
  <span class="ca-label">$BLK</span>
  <span class="ca-address" id="contract-address">TBA — Contract address will be
  <button class="ca-copy-btn" onclick="copyCA()">
    <svg><!-- clipboard icon --></svg>
  </button>
</div>
```

Styling:

- `.ca-label` : color: #c9a96e, font-weight: 700, font-size: 13px, margin-right: 12px
- `.ca-address` : font-family: 'JetBrains Mono', 'Courier New', monospace; font-size: 12px; color: #ffffff; letter-spacing: 0.5px; user-select: all
- `.ca-copy-btn` : background: transparent, border: 1px solid #2a2a2a, border-radius: 4px, padding: 4px 8px, cursor: pointer, color: #888, transition: all 0.2s

- `.ca-copy-btn:hover` : border-color: #c9a96e, color: #c9a96e

Copy functionality:

- On click, copy the address text to clipboard
- Change button text/icon to a checkmark and show "Copied!" for 2 seconds
- Then revert back to clipboard icon

Pre-launch state: Show "TBA — Contract address will be published at launch" in the address field. The copy button should be disabled/hidden until a real address is set.

Mobile: Full width, text truncates in the middle with ellipsis if needed (show first 6 and last 6 chars). Copy button still works.

2. BACKEND — Express.js API + SQLite + Cron

Create a backend server that powers the live stats. This runs alongside the existing frontend.

Environment Variables (store in Replit Secrets):

Plain Text

```
HELIUS_API_KEY=your_helius_api_key_here
BLK_MINT_ADDRESS=TBA
FUND_WALLET_ADDRESS=your_fund_wallet_public_key
MINIMUM_BLK_BALANCE=1000000
```

Database Schema (SQLite):

SQL

```
CREATE TABLE IF NOT EXISTS snapshots (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  snapshot_id TEXT UNIQUE NOT NULL,
  timestamp TEXT NOT NULL,
  total_holders INTEGER,
  eligible_holders INTEGER,
  total_supply REAL
);

CREATE TABLE IF NOT EXISTS holder_records (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  snapshot_id TEXT NOT NULL,
  wallet TEXT NOT NULL,
  balance REAL NOT NULL,
```

```

FOREIGN KEY (snapshot_id) REFERENCES snapshots(snapshot_id)
);

CREATE TABLE IF NOT EXISTS distributions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  operation_id TEXT UNIQUE NOT NULL,
  token_name TEXT,
  token_mint TEXT,
  total_distributed REAL,
  total_recipients INTEGER,
  timestamp TEXT,
  tx_signatures TEXT,
  status TEXT DEFAULT 'completed'
);

CREATE TABLE IF NOT EXISTS stats_cache (
  key TEXT PRIMARY KEY,
  value TEXT,
  updated_at TEXT
);

CREATE INDEX IF NOT EXISTS idx_holder_wallet ON holder_records(wallet);
CREATE INDEX IF NOT EXISTS idx_holder_snapshot ON holder_records(snapshot_id);

```

API Endpoints:

GET /api/stats

Returns current fund stats for the Vault page:

JSON

```

{
  "fundWalletBalance": 12.5,
  "nextDistributionPool": "Accumulating",
  "totalDistributed": 0,
  "totalDistributedUSD": "$0",
  "eligibleHolders": 0,
  "totalHolders": 0,
  "operationsCompleted": 0,
  "lastUpdated": "2025-01-15T10:30:00Z",
  "status": "pre-launch"
}

```

Pre-launch behavior: If BLK_MINT_ADDRESS is "TBA" or empty, return a pre-launch response with all zeros and status "pre-launch". The frontend shows clean placeholder states.

GET /api/tiers

Returns tier breakdown for pie chart:

JSON

```
{
  "tiers": [
    { "name": "Tier 1 (Top 10)", "holders": 10, "percentage": 45.2, "color": "#"},
    { "name": "Tier 2 (Top 11-50)", "holders": 40, "percentage": 28.7, "color": "#"},
    { "name": "Tier 3 (Top 51-200)", "holders": 150, "percentage": 18.4, "color": "#"},
    { "name": "Tier 4 (Remaining)", "holders": 287, "percentage": 7.7, "color": "#"}
  ],
  "totalEligible": 487,
  "lastUpdated": "2025-01-15T10:30:00Z"
}
```

GET /api/holder/:address

Returns info for a specific connected wallet:

JSON

```
{
  "wallet": "7xKXtg...",
  "balance": 5000000,
  "tier": 2,
  "tierName": "Tier 2 (Top 11-50)",
  "loyaltyWeeks": 4,
  "loyaltyMultiplier": 1.5,
  "isDiamondHands": true,
  "estimatedAllocation": 2.34,
  "rank": 27,
  "eligible": true
}
```

If wallet not found or below minimum: { "eligible": false, "reason": "Below minimum threshold" }

GET /api/distributions

Returns distribution history:

JSON

```
{
  "distributions": [
    {
      "operationId": "OP-001",
      "tokenName": "TARGETTOKEN",

```

```

    "totalDistributed": 50000000,
    "recipients": 487,
    "timestamp": "2025-01-15T14:00:00Z",
    "txSignatures": ["5UxM7...", "3KpL9..."],
    "status": "completed"
  }
]
}

```

Cron Job — Periodic Snapshots:

JavaScript

```

const cron = require('node-cron');

// Run snapshot every 4 hours
cron.schedule('0 */4 * * *', async () => {
  try {
    if (process.env.BLK_MINT_ADDRESS === 'TBA') {
      console.log('Pre-launch mode – skipping snapshot');
      return;
    }
    await takeSnapshot();
    await updateStatsCache();
    console.log(`Snapshot completed at ${new Date().toISOString()}`);
  } catch (error) {
    console.error('Cron snapshot failed:', error.message);
    // Don't crash – just log and try again next cycle
  }
});

```

Error Handling Rules:

- If Helius API fails: serve last cached data from stats_cache table. Never return an error to the frontend.
- If database is empty (pre-launch): return clean zero/placeholder responses.
- All API endpoints wrapped in try/catch — always return valid JSON, never crash.
- Add a `"dataFresh"` boolean to responses: true if data is < 5 hours old, false if stale.

3. LIVE STATS ON THE VAULT PAGE

Replace the existing placeholder dashes on the Vault page with live data from the backend.

Layout: Keep the existing 4-stat grid layout. Update it to:

Plain Text

Fund Balance 12.5 SOL	Distribution Pool Status Accumulating	Total Distributed \$0	Eligible Holders 0
--------------------------	---	-----------------------------	--------------------------

Last updated: 3 minutes ago

Behavior:

- On page load: fetch GET /api/stats
- Poll every 5 minutes (setInterval)
- Numbers animate in with a subtle count-up effect
- "Last updated: X minutes ago" text below the stats bar (small, muted gray, #666)
- If fetch fails: keep showing last known values, change "Last updated" to "Updating..." in slightly dimmer text
- Pre-launch state: show "—" for all values with "Live data available post-launch" subtitle

Stat formatting:

- Fund Balance: show as "X.XX SOL" (gold color for the number)
- Distribution Pool: show status text like "Accumulating" or "Ready: X tokens"
- Total Distributed: show as dollar value "\$X,XXX" or "—" if zero
- Eligible Holders: show as integer with comma formatting

4. PIE/DONUT CHART — Distribution Allocation by Tier

Add a donut chart below the stats section on the Vault page.

Section layout:

- Headline: "Shareholder Allocation Breakdown" (Playfair Display, white)
- Subtitle: "Distribution weight by holder tier" (Inter, muted gray)
- Chart on the left (60% width on desktop), legend on the right (40%)
- On mobile: chart full width, legend below

Chart specs:

- Use Chart.js (lightweight, easy to implement) with doughnut type

- Colors: gold gradient tones matching the theme:
 - Tier 1: #c9a96e (bright gold)
 - Tier 2: #a08540 (medium gold)
 - Tier 3: #6b5a2e (dark gold)
 - Tier 4: #3d3420 (deep bronze)
- Chart background: transparent
- Center text inside donut: total eligible holders count
- Smooth animation on load
- Hover: show tier name + exact percentage + holder count

Legend (right side):

Plain Text		
• Tier 1 – Top 10 Holders		45.2%
• Tier 2 – Top 11-50 Holders		28.7%
• Tier 3 – Top 51-200 Holders		18.4%
• Tier 4 – Remaining Eligible		7.7%

Pre-launch state: Show the chart with even 25% splits and a note: "Allocation data will populate once \$BLK is live and holder snapshots begin."

Data source: GET /api/tiers — poll same interval as stats (every 5 min)

5. WALLET CONNECT — "Check Your Allocation"

Add a section below the pie chart on the Vault page.

Section headline: "Check Your Allocation"

Subtitle: "Connect your wallet to view your shareholder status and estimated distribution weight."

Button: "Connect Wallet" — styled as outlined button with gold border, gold text. On hover: filled gold with black text.

Implementation: Use @solana/wallet-adapter-react and @solana/wallet-adapter-wallets (Phantom primarily). Standard Solana wallet connect flow.

Connected State — Show a card/panel with:

Plain Text

Your Shareholder Status

Wallet: 7xKX...gAsU

\$BLK Balance: 5,000,000

Tier: 2 (Top 11-50)

Loyalty: 4 weeks (1.5x multiplier)

Diamond Hands: ✓ Yes (1.5x bonus)

Estimated Allocation: 2.34%

Rank: #27 of 487 eligible

[Disconnect]

Styling:

- Card background: #141414
- Border: 1px solid #1f1f1f
- Labels in muted gray, values in white
- Tier and multiplier values in gold
- Diamond hands checkmark in gold
- Disconnect button: small, subtle, bottom of card

Not eligible state:

Plain Text

Wallet: 7xKX...gAsU

You are not currently a qualifying shareholder.

Minimum holding: 100,000 \$BLK

[Acquire \$BLK →]

[Disconnect]

Pre-launch state: Button still works for connection, but after connecting show:

"Shareholder data will be available once \$BLK launches and snapshots begin. Your wallet is connected and ready."

Data source: After wallet connects, call GET /api/holder/:address with the connected wallet's public key.

6. DISTRIBUTION HISTORY ENHANCEMENT

Update the existing Distribution History table on the Vault page:

Fetch data from: GET /api/distributions

Table columns:

Operation	Asset	Distributed	Recipients	Date	Verify
-----------	-------	-------------	------------	------	--------

"Verify" column: Each row has a "View on Solscan" link that opens the transaction signature on solscan.io. If multiple transactions per operation, show "View X transactions" that expands to show all signatures.

Link format: `https://solscan.io/tx/{signature}`

Empty state (pre-launch):

"No distributions yet. The first operation is pending. All future distributions will be verifiable on-chain with direct Solscan links."

7. ADDITIONAL NOTES

- **DO NOT** break or restyle any existing pages/sections. Only ADD the new features described above.
- The CA bar goes on ALL pages (it's a global component in the layout).
- The live stats, pie chart, wallet connect, and distribution history enhancements are all on the VAULT page only.
- All new UI must be mobile responsive — test on 375px width (iPhone SE) and 390px (iPhone 14).
- Use loading skeletons (subtle pulse animation on dark cards) while data is being fetched — never show blank/broken states.
- If any API call fails, show the last known good state or a clean "—" placeholder. NEVER show error messages, stack traces, or "undefined" to users.
- Install required packages: chart.js, react-chartjs-2, @solana/wallet-adapter-react, @solana/wallet-adapter-wallets, @solana/wallet-adapter-react-ui, @solana/web3.js, better-sqlite3, node-cron, express, cors, axios
- Store HELIUS_API_KEY, BLK_MINT_ADDRESS, and FUND_WALLET_ADDRESS in Replit Secrets (environment variables).